LAB REPORT 08
Circuit Analysis and Filter Design

Example 01: Diode I-V Characteristics

> Calculates and plots the current-voltage (I-V) characteristics of a diode from -0.7 V to +0.7 V.

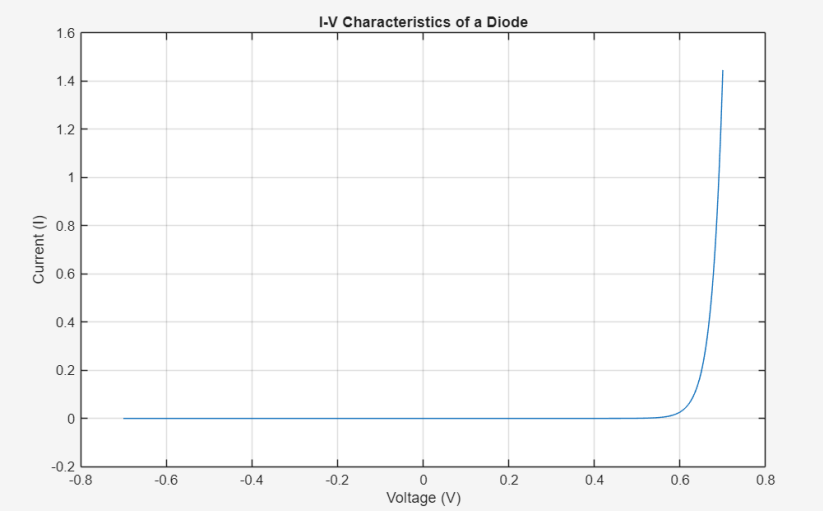
MATLAB Implementation Code:

```
% Step 1: Define parameters and voltage range
Is = 1e-12; % Saturation current in amperes
Vt = 0.025; % Thermal voltage in volts
V = linspace(-0.7, 0.7, 1000); % Applied voltage sweep

% Step 2: Calculate diode current using Shockley equation
I = Is * (exp(V / Vt) - 1);

% Step 3: Plot I-V characteristics
plot(V, I);
xlabel('Voltage (V)');
ylabel('Current (I)');
title('I-V Characteristics of a Diode');
grid on;
```

Simulation Output:



Example 02: RL High-Pass Filter Frequency Response

> Calculates the transfer function of an RL high-pass filter and plots its Bode plot.

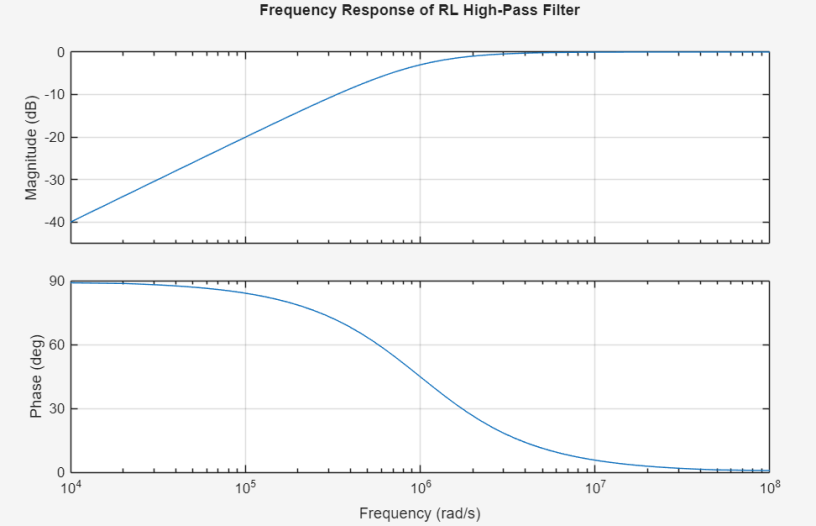
MATLAB Implementation Code:

```
% Step 1: Define circuit components
R = 1e3; % Resistance in ohms
L = 1e-3; % Inductance in henries

% Step 2: Calculate transfer function H(s) = Ls / (Ls + R)
H = tf([L 0], [L R]);

% Step 3: Plot Bode frequency response
bode(H);
title('Frequency Response of RL High-Pass Filter');
grid on;
```

Simulation Output:



Example 03: Series RLC Circuit Step Response

> Creates the transfer function of a series RLC circuit and plots its step response over time.

MATLAB Implementation Code:

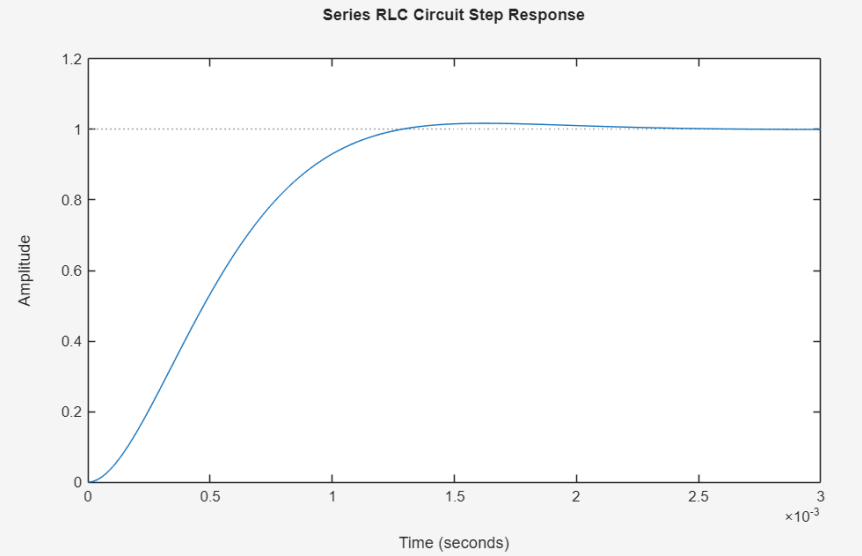
```
% Step 1: Define circuit components
R = 5;       % Resistance in ohms
L = 1e-3;    % Inductance in henrys
C = 100e-6;  % Capacitance in farads

% Step 2: Calculate transfer function H(s) = 1 / (LC*s^2 + RC*s + 1)
```

```
sys = tf([1], [L*C R*C 1]);

% Step 3: Plot step response
step(sys);
title('Series RLC Circuit Step Response');
```

Simulation Output:



Example 04: BJT Common-Emitter Amplifier Analysis

> Calculates the collector-emitter voltage (V_CE) of a BJT amplifier.

MATLAB Implementation Code:

```
% Step 1: Define transistor parameters
beta = 100;     % Current gain
R_C = 1e3;      % Collector resistor in ohms
V_CC = 10;      % Supply voltage in volts
I_B = 50e-6;    % Base current in amperes

% Step 2: Calculate currents and voltages
I_C = beta * I_B;          % Collector current
V_CE = V_CC - I_C * R_C;   % Collector-emitter voltage

% Step 3: Display result
disp(['Collector-Emitter Voltage: ', num2str(V_CE), ' V']);
```

Simulation Output:

```
Command Window
>> example_04
Collector-Emitter Voltage: 5 V
>>
```

Example 05: FIR Bandpass Filter Design

> Designs a 50th-order FIR bandpass filter for frequencies between 100 Hz and 200 Hz.

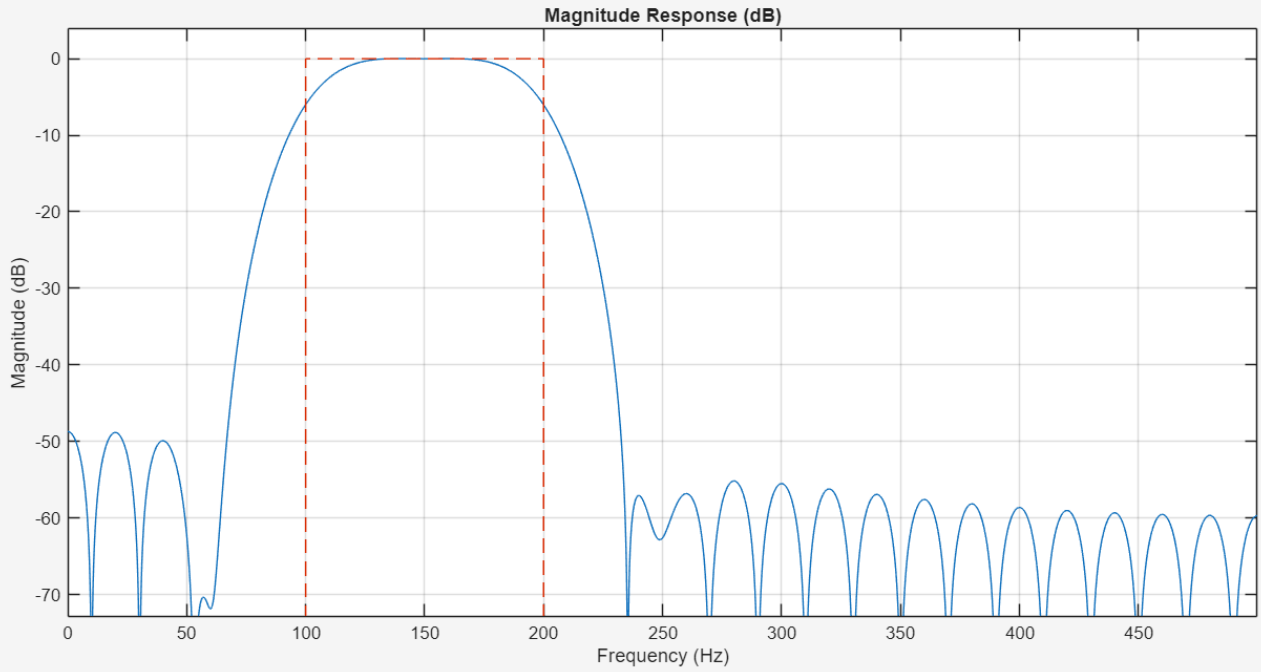
MATLAB Implementation Code:

```
% Step 1: Define filter parameters
Fs = 1000;     % Sampling frequency in Hz
Fpass1 = 100;  % First passband cutoff frequency in Hz
Fpass2 = 200;  % Second passband cutoff frequency in Hz
N = 50;        % Filter order

% Step 2: Design FIR bandpass filter
bpFilt = designfilt('bandpassfir', 'FilterOrder', N, ...
    'CutoffFrequency1', Fpass1, 'CutoffFrequency2', Fpass2, ...
    'SampleRate', Fs);

% Step 3: Visualize filter frequency response
fvtool(bpFilt);
title('Bandpass Filter Design');
```

Simulation Output:



Example 06: Op-Amp Inverting Amplifier

> Calculates the output voltage of an inverting operational amplifier.

MATLAB Implementation Code:

```
% Step 1: Define circuit components and input
Rin = 10e3;   % Input resistor in ohms
Rf = 100e3;   % Feedback resistor in ohms
Vin = 1;      % Input voltage in volts

% Step 2: Calculate voltage gain and output voltage
Av = -Rf / Rin;     % Voltage gain (-10)
Vout = Av * Vin;    % Output voltage

% Step 3: Display result
disp(['Output Voltage: ', num2str(Vout), ' V']);
```

Simulation Output:

```
Command Window
>> example_06
Output Voltage: -10 V
>>
```

Example 07: RC Circuit Transient Impulse Response

> Creates the transfer function of an RC low-pass filter and plots its transient response to an impulse input.

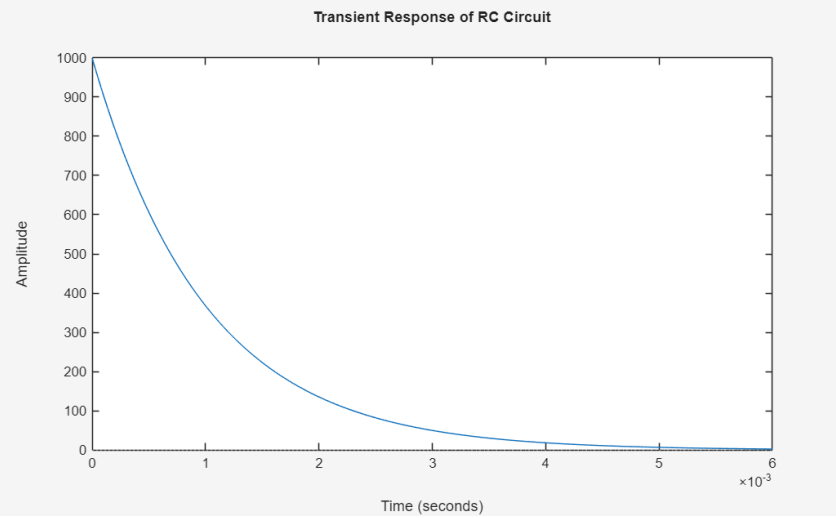
MATLAB Implementation Code:

```
% Step 1: Define circuit components
R = 1e3;   % Resistance in ohms
C = 1e-6;  % Capacitance in farads

% Step 2: Define transfer function H(s) = 1 / (RC*s + 1)
sys = tf([1], [R*C 1]);

% Step 3: Plot impulse response
impulse(sys);
title('Transient Response of RC Circuit');
```

Simulation Output:



Example 08: RLC Circuit Impedance Calculation

> Calculates total magnitude of impedance (Z) for a series RLC circuit at 60 Hz.

MATLAB Implementation Code:

```
% Step 1: Define parameters
R = 50;        % Resistance in ohms
L = 100e-3;    % Inductance in henrys
C = 10e-6;     % Capacitance in farads
f = 60;        % Frequency in Hz

% Step 2: Compute angular frequency and complex impedance
omega = 2 * pi * f;
Z = R + 1i * (omega * L - 1 / (omega * C));

% Step 3: Display total impedance magnitude
disp(['Impedance: ', num2str(abs(Z)), ' Ohms']);
```

Simulation Output:

```
Command Window
>> example_08
Impedance: 232.9875 Ohms
>>
```

Example 09: Schmitt Trigger Simulation

> Calculates switching threshold voltages and simulates output voltage behavior across an input voltage sweep.

MATLAB Implementation Code:

```
% Step 1: Define circuit parameters
Vcc = 5;      % Supply voltage in volts
R1 = 1e3;     % Resistor R1 in ohms
R2 = 10e3;    % Resistor R2 in ohms

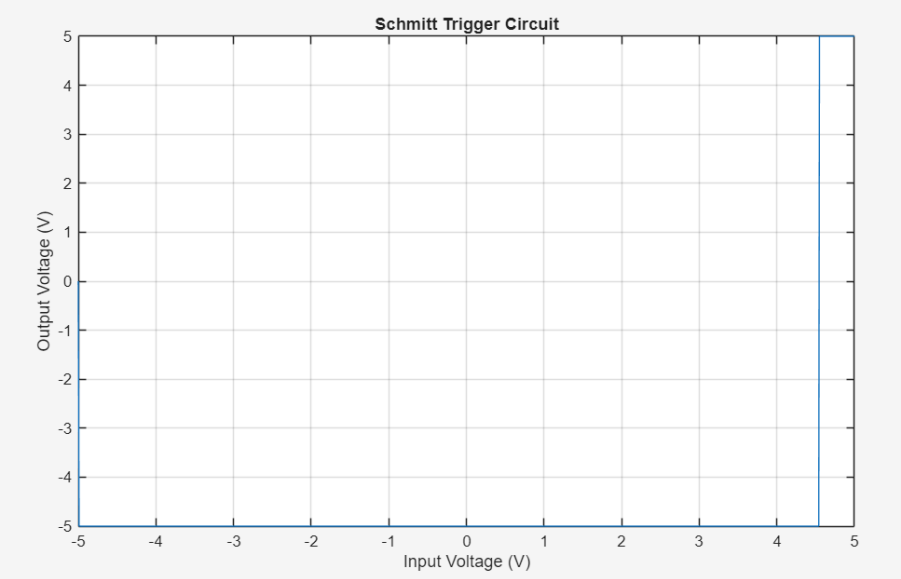
% Step 2: Calculate threshold voltages
Vh = Vcc * (R2 / (R1 + R2));   % High threshold voltage (+4.545 V)
Vl = -Vcc * (R2 / (R1 + R2));  % Low threshold voltage (-4.545 V)

% Step 3: Set up voltage sweep and initialize output
Vin = linspace(-Vcc, Vcc, 1000);
Vout = zeros(size(Vin));

% Step 4: Simulate hysteresis behavior
for i = 2:length(Vin)
    if Vin(i) > Vh
        Vout(i) = Vcc;
    elseif Vin(i) < Vl
        Vout(i) = -Vcc;
    else
        Vout(i) = Vout(i-1);
    end
end

% Step 5: Plot Schmitt trigger input vs output
plot(Vin, Vout);
title('Schmitt Trigger Circuit');
xlabel('Input Voltage (V)');
ylabel('Output Voltage (V)');
grid on;
```

Simulation Output:



Example 10: Full-Wave Rectifier Simulation

> Simulates a 50 Hz full-wave rectifier converting an AC sine wave into pulsating DC.

MATLAB Implementation Code:

```
% Step 1: Create time vector and AC input voltage wave
t = linspace(0, 0.1, 1000);           % Time duration of 0.1s
V_in = sin(2 * pi * 50 * t);          % 50 Hz AC signal

% Step 2: Perform full-wave rectification
V_out = abs(V_in);

% Step 3: Plot input vs output waveforms
plot(t, V_in, t, V_out);
title('Full-Wave Rectifier Simulation');
xlabel('Time (s)');
ylabel('Voltage (V)');
legend('Input Voltage', 'Output Voltage');
grid on;
```

Simulation Output:

